

Practice Sheet on View:

LAB ASSIGNMENT:

Sample table: salesman

salesman_id	name	city	commission
5001	James Hoog	New York	0.15
5002	Nail Knite	Paris	0.13
5005	Pit Alex	London	0.11
5006	Mc Lyon	Paris	0.14
5003	Lauson Hen	Berlin	0.12
5007	Paul Adam	Rome	0.13

Sample table: customer

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3005	Graham Zusi	California	200	5002
3001	Brad Guzan	London	300	5005
3004	Fabian Johns	Paris	300	5006
3007	Brad Davis	New York	200	5001
3009	Geoff Camero	Berlin	100	5003
3008	Julian Green	London	300	5002
3003	Jozy Altidor	Moscow	200	5007

Sample table: orders

ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.5	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.5	2012-08-17	3009	5003
70007	948.5	2012-09-10	3005	5002
70005	2400.6	2012-07-27	3007	5001
70008	5760	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.4	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.6	2012-04-25	3002	5001

1. Write a query to create a view for those salesmen belongs to the city New York.
2. Write a query to create a view for all salesmen with columns salesman_id, name and city.
3. Write a query to find the salesmen of the city New York who achieved the commission more than 13%.
4. Write a query to create a view to getting a count of how many customers we have at each level of a grade.
5. Write a query to create a view to keeping track the number of customers ordering, number of salesmen attached, average amount of orders and the total amount of orders in a day.
6. Write a query to create a view that shows for each order the salesman and customer by name.
7. Write a query to create a view that finds the salesman who has the customer with the highest order of a day.
8. Write a query to create a view that finds the salesman who has the customer with the highest order at least 3 times on a day.
9. Write a query to create a view that shows all of the customers who have the highest grade.
10. Write a query to create a view that shows the number of the salesman in each city.
11. Write a query to create a view that shows the average and total orders for each salesman after his or her name. (Assume all names are unique)
12. Write a query to create a view that shows each salesman with more than one customers.

13. Write a query to create a view that shows all matches of customers with salesman such that at least one customer in the city of customer served by a salesman in the city of the salesman.

14. Write a query to create a view that shows the number of orders in each day.

15. Write a query to create a view that finds the salesmen who issued orders on October 10th, 2012.

16. Write a query to create a view that finds the salesmen who issued orders on either August 17th, 2012 or October 10th, 2012.

Practice Sheet on Indexing and Cascading:

Sample Table – Worker

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
1	Rana	Hamid	100000	2014-02-20 09:00:00	HR
2	Sanjoy	Saha	80000	2014-06-11 09:00:00	Admin
3	Mahmudul	Hasan	300000	2014-02-20 09:00:00	HR
4	Asad	Zaman	500000	2014-02-20 09:00:00	Admin
5	Sajib	Mia	500000	2014-06-11 09:00:00	Admin
6	Alamgir	Kabir	200000	2014-06-11 09:00:00	Account
7	Foridul	Islam	75000	2014-01-20 09:00:00	Account
8	Keshob	Ray	90000	2014-04-11 09:00:00	Admin

1. Write the SQL query to create a non-unique index named `idx_worker_salary` on the `SALARY` column of the `Worker` table to speed up payroll queries.
2. Write the SQL command to create a composite index named `idx_name_dept` on the `Worker` table using the `FIRST_NAME` and `DEPARTMENT` columns.
3. Write the SQL statement to create a **unique** index named `idx_worker_id_unique` on the `WORKER_ID` column.
4. Provide the SQL command required to delete an index named `idx_worker_salary`.

Part 2: Cascading Tasks

1. Create a table named `Departments` with a Primary Key `DeptID`. Then, create a table named `Staff` where `DeptID` is a Foreign Key referencing the `Departments` table. Ensure that if a department is deleted, all staff members belonging to that department are also deleted from the `Staff` table.
2. Given an existing table `Orders` with a column `CustomerID`, write the SQL statement to add a Foreign Key constraint named `fk_cust_update` that references a `Clients` table. The constraint must ensure that if a `CustomerID` is changed in the `Clients` table, it is automatically updated in the `Orders` table.
3. Create a child table named `Project_Tasks` that references a parent table `Projects`. The parent table has a composite Primary Key consisting of `ProjectID` and `DeptCode`. Write the SQL to define the foreign key in `Project_Tasks` that uses **both** columns with an `ON DELETE CASCADE` rule.

4. Write the SQL to create a Shipments table referencing a Warehouse table. The constraint must satisfy these two conditions:
- Allow the WarehouseID to be updated automatically if changed in the parent table (**Cascade**).
 - **Prevent** the deletion of a warehouse if it still has active records in the Shipments table (**Restrict**).